Sistema de Chat

17/04/2026

Sumário

1. Introdução 2
2. Ambiente de Desenvolvimento 2
3. Sistema Chat 3
Protocolo da Camada de Aplicação 3
Operação 1 — Entrada no sistema (login sem autenticação) 4
Operação 2 — Listar usuários online 4
Operação 3 — Chat privado 5
Operação 4 — Mensagem para o setor 5
Operação 5 — Aviso global 5
Operação 6 — Encerramento da conexão 6
Operação 7 — Arquitetura 6
Protocolo da Camada de Transporte 7
Protótipo e Funcionamento do Sistema 7
Execução do sistema 7
Funcionamento do servidor 8
Funcionamento do cliente 9
4. Conclusão 11
5. Referencias 11

1. Introdução

A metodologia adotada neste trabalho consistiu na implementação prática de um sistema de comunicação em rede baseado no modelo cliente-servidor, utilizando sockets na linguagem Python. Inicialmente, foi desenvolvido o servidor, responsável por estabelecer conexões com múltiplos clientes por meio do protocolo TCP. Para permitir o atendimento simultâneo de diversas conexões, foram utilizados threads, possibilitando que cada cliente fosse tratado de forma independente e concorrente.

Em seguida, foi implementado o cliente, capaz de se conectar ao servidor e realizar o envio e recebimento de mensagens conforme o protocolo de aplicação definido. Para garantir a comunicação contínua e não bloqueante, também foram utilizados threads no lado do cliente, permitindo que o envio de mensagens pelo usuário e a recepção de dados do servidor ocorressem de forma paralela.

Além disso, foi definido um protocolo de aplicação baseado em mensagens no formato JSON, possibilitando a padronização da comunicação entre cliente e servidor e facilitando a interpretação das operações realizadas.

Durante o desenvolvimento, foram realizados testes em ambiente local (localhost), com múltiplas instâncias de clientes conectadas ao servidor, a fim de validar o estabelecimento de conexões, a troca de mensagens em tempo real e o correto funcionamento das funcionalidades implementadas. Também foram considerados cenários de erro, como desconexões inesperadas e falhas de comunicação, visando aumentar a robustez e a confiabilidade do sistema.

2. Ambiente de Desenvolvimento

O desenvolvimento do sistema foi realizado em um mesmo ambiente de rede local, utilizando a máquina local para execução tanto do cliente quanto do servidor. O código foi desenvolvido no editor Visual Studio Code, enquanto a execução dos programas foi feita através do terminal integrado. Dessa forma, ambos os lados da comunicação (cliente e servidor) operaram na mesma rede, facilitando os testes e a verificação do funcionamento do protocolo.

3. Sistema Chat

3.1 Perguntas

3.1.1 Defina o protocolo de aplicação que você utilizará para criação de uma aplicação de chat. Para isso, faço uma lista de operações que o cliente deseja realizar e para cada operação você deverá pensar como seria o protocolo de aplicação que o ajudaria a realizar essas operações.

Protocolo da Camada de Aplicação

Neste sistema, foi utilizado o formato JSON que permite representar dados de forma estruturada e legível.

Tabela 1- Protocolo de Aplicação (mensagens JSON)

Operação	Direção	Mensagem JSON
Login	Cliente → Servidor	{"type": "login", "user": "nome", "area": 1}
	Servidor → Cliente	{"type": "login_ok"}
	Servidor → Cliente	{"type": "erro", "msg": "nome_em_uso"}
Listar online	Cliente → Servidor	{"type": "list"}
	Servidor → Cliente	{"type": "online", "users": [{"user": "nome", "area": 1}]}
Convite de chat	Cliente → Servidor	{"type": "chat_request", "from": "A", "to": "B"}
	Servidor → Cliente	{"type": "chat_invite", "from": "A"}
Aceitar chat	Cliente → Servidor	{"type": "chat_accept", "from": "B", "to": "A"}
	Servidor → Cliente	{"type": "chat_accept", "from": "B", "to": "A"}
Recusar chat	Cliente → Servidor	{"type": "chat_reject", "from": "B", "to": "A"}
	Servidor → Cliente	{"type": "chat_reject", "from": "B", "to": "A"}
Mensagem privada	Cliente → Servidor	{"type": "mensagem", "from": "A", "to": "B", "text": "Oi"}
	Servidor → Cliente	{"type": "mensagem", "from": "A", "to": "B", "text": "Oi"}
Mensagem para setor	Cliente → Servidor	{"type": "broadcast", "from": "A", "area": 1, "text": "Aviso"}
	Servidor → Clientes	{"type": "broadcast", "from": "A", "area": 1, "text": "Aviso"}
Aviso global	Cliente → Servidor	{"type": "aviso", "from": "A", "area": 1, "text": "Aviso geral"}
	Servidor → Clientes	{"type": "aviso", "from": "A", "area": 1, "text": "Aviso geral"}
Erro	Servidor → Cliente	{"type": "erro", "msg": "descricao_do_erro"}

Operação 1 — Entrada no sistema (login sem autenticação)

Para melhor organização da rede o cliente envia seu nome e departamento de trabalho para o servidor e o servidor valida se o nome já está em uso e responde confirmando ou rejeitando o acesso.

```
Cliente → Servidor:
{"type": "login", "user": "user1", "area": 1}
```

```
Servidor → Cliente (sucesso):
{"type": "login_ok"}
```

```
Servidor → Cliente (erro):
{"type": "erro", "msg": "nome_em_uso"}
```

Operação 2 — Listar usuários online

O cliente pode solicitar ao servidor a lista de usuários conectados no momento. Além disso, o servidor também envia essa lista automaticamente sempre que ocorre alguma alteração, como a entrada ou saída de usuários. Os usuários não possuem um identificador fixo, apenas um "nickname" e uma área associada. Essas informações podem mudar a cada nova conexão com o servidor, já que são atribuídas dinamicamente quando o usuário entra no sistema.

```
Cliente → Servidor:
{"type": "list"}
```

```
Servidor → Cliente:
{"type": "online", "users": [{"user": "user1", "area": 3}, {"user": "duda", "area": 2}]}
```

Operação 3 — Chat privado

A comunicação privada ocorre em três etapas: convite, resposta e troca de mensagens. O servidor atua como intermediário entre os clientes.

```
Convite:
A → Servidor: {"type": "chat_request", "from": "laila", "to": "duda"}
Servidor → B: {"type": "chat_invite", "from": "laila"}
```

```
Resposta:
B → Servidor: {"type": "chat_accept", "from": "duda", "to": "laila"}
Servidor → A: {"type": "chat_accept", "from": "duda", "to": "laila"}
```

Troca de mensagens:
A → Servidor: {"type": "message", "from": "laila", "to": "duda", "text": "Oi!"}
Servidor → B: {"type": "message", "from": "laila", "to": "duda", "text": "Oi!"}

Operação 4 — Mensagem para o setor

Nesta operação, o cliente envia uma mensagem destinada a todos os usuários que pertencem à mesma área. Embora esse comportamento seja descrito na aplicação como "broadcast", ele não corresponde ao broadcast real utilizado em redes de computadores.

Em redes, o broadcast ocorre na camada de rede e consiste no envio de uma mensagem para todos os dispositivos de uma rede. Já o multicast permite o envio para um grupo específico de dispositivos, sendo geralmente associado ao uso do protocolo UDP.

No sistema desenvolvido, que utiliza o protocolo TCP, esse tipo de envio não é suportado diretamente. Isso ocorre porque o TCP estabelece conexões ponto a ponto, ou seja, entre dois dispositivos apenas. Dessa forma, o envio para múltiplos usuários é implementado na camada de aplicação. O cliente envia uma única mensagem ao servidor, e o servidor é responsável por identificar todos os usuários da mesma área e encaminhar a mensagem individualmente para cada um deles. Por esse motivo, o comportamento pode ser entendido como um "broadcast simulado" ou multicast implementado na aplicação.

Assim, todos os clientes pertencentes à mesma área recebem a mensagem sem necessidade de envio direto entre os clientes.

Cliente → Servidor: {"type": "broadcast", "from": "laila", "area": 3, "text": "Reunião em 10 min"}

Operação 5 — Aviso global

Neste protocolo, o cliente envia uma mensagem destinada a todos os usuários conectados ao sistema, independentemente da área.

O cliente realiza uma requisição ao servidor, que então redistribui a mensagem para todos os usuários ativos. Como o aviso é global, não é necessário incluir a área no JSON, pois a mensagem não é filtrada por grupo.

Cliente → Servidor: {"type": "aviso", "from": "laila", "area": 3, "text": "Sistema em manutenção"}

Operação 6 — Encerramento da conexão

Não existe uma mensagem explícita de logout no protocolo. O encerramento da conexão ocorre quando o cliente finaliza a aplicação ou fecha a conexão TCP. O servidor detecta automaticamente

essa desconexão e remove o usuário da lista de conectados, atualizando a lista de usuários online para os demais clientes.

Além disso, no sistema existem duas formas de saída em nível de aplicação:

- O comando `/sair`, utilizado para encerrar uma conversa específica.
- E a opção "sair" no menu, que encerra a execução do cliente e, consequentemente, a conexão com o servidor.

Dessa forma, a remoção do usuário ocorre de forma automática pelo servidor, sem necessidade de envio de uma mensagem específica de logout.

Operação 7 — Arquitetura

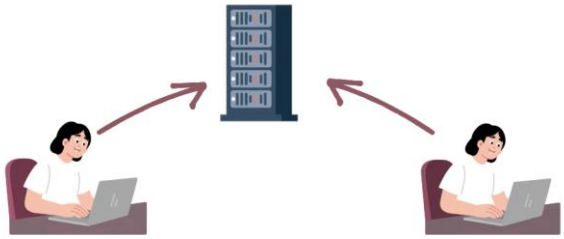

Figura 1- Arquitetura do servidor

A arquitetura do sistema segue o modelo cliente-servidor, no qual múltiplos clientes se conectam a um servidor central responsável por gerenciar as conexões e intermediar a troca de mensagens. Nesse modelo, os clientes não se comunicam diretamente entre si, sendo toda a comunicação realizada por meio do servidor. Isso pode ser observado na implementação dos protocolos, em que todas as mensagens passam obrigatoriamente pelo servidor antes de serem encaminhadas ao cliente de destino.

3.1.1 Qual protocolo da camada de transporte você utilizará para sua aplicação de chat? Descreva as características do protocolo na camada de transporte que você escolheu e justifique porque fez essa escolha.

Protocolo da Camada de Transporte

A aplicação utiliza o protocolo TCP (Transmission Control Protocol), que é um protocolo orientado à conexão. Isso significa que antes da troca de dados é estabelecida uma conexão entre cliente e servidor por meio do processo conhecido como handshake de três vias.

O TCP oferece características fundamentais para o funcionamento do chat:

Entrega confiável: garante que os dados enviados cheguem ao destino, retransmitindo pacotes perdidos.
Entrega ordenada: assegura que os dados cheguem na mesma ordem em que foram enviados.
Controle de fluxo e congestionamento: ajusta a taxa de envio de acordo com a capacidade da rede.

Essas propriedades são essenciais para aplicações de chat, pois não é aceitável que mensagens sejam perdidas ou cheguem fora de ordem. Diferentemente do TCP, o protocolo UDP não garante entrega nem ordenação, sendo mais rápido, porém inadequado para este tipo de aplicação.

3.1.1 Escreva um protótipo do seu chat, de preferência usando a linguagem Python, que comporte a troca de mensagens de texto entre, no mínimo, dois clientes. Descreva como seu código funciona em detalhes, e como executá-lo.

Protótipo e Funcionamento do Sistema

Execução do sistema

Para executar o sistema, é necessário ter o Python 3 instalado. O servidor e os clientes devem ser executados em terminais separados:

```
# Terminal 1 — servidor
python servidor.py

# Terminal 2 — cliente
python cliente.py

# Terminal 3 — outro cliente
python cliente.py

...

# Terminal n — n cliente
python cliente.py
```

Por padrão, todos executam na mesma máquina (127.0.0.1).

Funcionamento do servidor

O servidor é o componente central do sistema e segue o modelo cliente-servidor, sendo responsável por gerenciar as conexões e intermediar toda a comunicação entre os clientes. Ele cria um socket TCP associado ao endereço `0.0.0.0` e à porta 5050, permitindo aceitar conexões provenientes de qualquer interface de rede da máquina. Em seguida, entra em um laço infinito aguardando novas conexões por meio da chamada `accept()`.

Sempre que um novo cliente se conecta, o servidor cria um thread **dedicada** para tratar essa conexão, executando a função `handle`. Essa abordagem permite que múltiplos clientes sejam atendidos simultaneamente, sem que a comunicação de um usuário bloqueie a dos demais, caracterizando um modelo concorrente.

O estado da aplicação é mantido por meio de duas estruturas principais: um dicionário que associa o nome do usuário ao seu socket de conexão (`clientes`) e outro que armazena a área correspondente de cada usuário (`areas`). Como essas estruturas são acessadas por múltiplas threads simultaneamente, é utilizado um mecanismo de sincronização (**lock**) para garantir exclusão mútua, evitando inconsistências e condições de corrida.

Dentro da função `handle`, o servidor permanece em loop recebendo dados do cliente através do método `recv()`. Cada mensagem recebida é decodificada e interpretada como um objeto JSON, sendo analisado o campo `"type"` para determinar a ação a ser executada. Esse mecanismo implementa o protocolo de aplicação definido para o sistema. Uma coisa para adicionar em relação a essa aplicação é que a decodificação é limitada

```
def handle(conn):
    nome = None
    try:
        while True:
            dados = conn.recv(4096).decode()
            if not dados:
                break
            msg = json.loads(dados)
            tipo = msg["type"]
```

Figura 2- Limitação bytes

Porém o TCP não envia mensagens separadas, ele envia tudo como um fluxo contínuo de bytes. Por isso, uma mensagem JSON pode chegar quebrada ou juntada com outras. No programa, os dados são lidos em blocos de 4096 bytes, então é necessário tratar na aplicação para saber onde cada mensagem começa e termina.

Isso ocorre de forma diferente do UDP por exemplo, que envia cada mensagem de forma separada, como um pacote. Se a mensagem for maior que o limite do buffer ou da rede, ela pode ser perdida ou chegar incompleta. Se algum pedaço do pacote se perder, a mensagem inteira não é recebida.

As principais operações tratadas pelo servidor incluem:

- **Login:** verifica se o nome de usuário já está em uso. Caso esteja disponível, adiciona o cliente às estruturas de controle e envia uma confirmação. Em seguida, atualiza todos os clientes com a nova lista de usuários online.
- **Listagem de usuários:** envia ao cliente a lista atualizada de usuários conectados.
- **Chat privado:** encaminha convites, aceitações e mensagens diretamente entre os clientes envolvidos.
- **Envio para o setor:** o servidor identifica os usuários pertencentes à mesma área e envia a mensagem individualmente para cada um, caracterizando um envio em grupo na camada de aplicação.
- **Aviso global:** a mensagem é encaminhada para todos os clientes conectados, sem distinção.

A comunicação é sempre mediada pelo servidor, ou seja, os clientes não se comunicam diretamente entre si. Para cada envio, o servidor utiliza o socket correspondente ao destinatário e transmite os dados no formato JSON.

Caso um cliente se desconecte o servidor detecta o encerramento quando a função `recv()` retorna vazio ou ocorre uma exceção. Nesse momento, o usuário é removido das estruturas internas e todos os clientes restantes são notificados com a lista atualizada de usuários online. Por fim, o socket da conexão é encerrado.

Além disso, o servidor utiliza a opção `SO_REUSEADDR`, que permite reutilizar rapidamente a porta após o encerramento do programa, facilitando a execução durante testes. Dessa forma, o servidor garante o gerenciamento centralizado das conexões, o controle do estado dos usuários e a entrega correta das mensagens, sendo o elemento fundamental para o funcionamento do sistema de comunicação.

Funcionamento do cliente

O cliente é responsável por estabelecer a conexão com o servidor e permitir a interação do usuário com o sistema de chat. Como a comunicação em rede envolve eventos assíncronos, isto é, mensagens podem chegar a qualquer momento, o cliente foi projetado para lidar simultaneamente com duas fontes de entrada: o teclado do usuário e os dados recebidos do servidor.

Um desafio importante nesse contexto é que operações como `input()` e `recv()` são bloqueantes, ou seja, interrompem a execução do programa até que uma entrada seja recebida. Para evitar que o sistema fique travado aguardando apenas uma dessas entradas, foi adotada uma abordagem baseada em concorrência com múltiplas threads.

Dessa forma, o cliente utiliza duas threads principais:

Thread de rede: responsável por ficar continuamente aguardando mensagens vindas do servidor por meio do socket (`sock.recv()`). Sempre que uma mensagem é recebida, ela é convertida do formato JSON para um objeto Python e inserida em uma fila (`fila_rede`). Caso a conexão seja encerrada, a thread sinaliza o evento ao sistema.

Thread de teclado: responsável por capturar a entrada do usuário através do terminal (`stdin`). Cada linha digitada é armazenada em outra fila (`fila_teclado`), permitindo que o restante do programa processe essa entrada posteriormente.

Esses threads não processam diretamente os dados; elas apenas alimentam filas. O processamento central ocorre em um **loop principal**, que executa continuamente verificando se há novos elementos nessas filas. Quando uma mensagem do servidor é encontrada, o cliente interpreta o campo `"type"` e executa a ação correspondente, como exibir mensagens, tratar convites de chat ou atualizar a lista de usuários online. Da mesma forma, quando há entrada do usuário, o sistema decide como interpretá-la com base no estado atual da aplicação.

Esse comportamento é controlado por uma variável chamada **contexto**, que representa o estado atual do cliente. Por exemplo, o cliente pode estar no menu principal (`"menu"`), em um chat ativo (`"chat:usuario"`), respondendo a um convite (`"convite:usuario"`) ou aguardando resposta de outro usuário (`"aguardando:usuario"`). Esse mecanismo permite que a mesma entrada do usuário seja interpretada de formas diferentes, dependendo do estado em que o cliente se encontra. Dessa forma, a interação se torna mais organizada e intuitiva, evitando conflitos como a sobreposição de mensagens ou interpretações incorretas dos comandos.

```python
def tratar_input(linha):
    global contexto

    if contexto.startswith("chat:"):
        com_quem = contexto.split(":", 1)[1]
        if linha.lower() == "/sair":
            print(f"[chat com {com_quem} encerrado]")
            contexto = "menu"
            prompt()
        elif linha:
            enviar({"type": "mensagem", "from": meu_nome, "to": com_quem, "text": linha})
            prompt()
        else:
            prompt()
        return

    if contexto.startswith("convite:"):
        de = contexto.split(":", 1)[1]
        if linha.lower() == "s":
            enviar({"type": "chat_accept", "from": meu_nome, "to": de})
            contexto = f"chat:{de}"
            print(f"[chat com {de} iniciado]")
            prompt()
        else:
            enviar({"type": "chat_reject", "from": meu_nome, "to": de})
            print("Recusado.")
            contexto = "menu"
            prompt()
        return

    if contexto.startswith("aguardando:"):
        return

    # menu
    if contexto == "menu":
        tratar_menu(linha)
```

Figura 2- Utilização da variável contexto

Além disso, o cliente mantém informações locais sobre a sessão, como o nome do usuário, sua área e a lista de usuários online, que são atualizadas dinamicamente conforme mensagens do servidor são recebidas.

Por fim, a comunicação com o servidor é realizada por meio do envio de mensagens no formato JSON através do socket TCP. Cada ação do usuário resulta no envio de uma mensagem estruturada, que é interpretada pelo servidor conforme o protocolo de aplicação definido visto anteriormente.

4. Conclusão

Este trabalho permitiu a aplicação prática dos conceitos de redes de computadores, especialmente o modelo cliente-servidor e o uso de sockets com o protocolo TCP. A implementação do chat possibilitou compreender como ocorre a comunicação entre múltiplos clientes, bem como o gerenciamento de conexões de forma concorrente.

O uso de JSON facilitou a padronização das mensagens, enquanto a utilização de threads garantiu a execução simultânea das operações. Além disso, foram exploradas características importantes do TCP, como a necessidade de tratar o fluxo contínuo de bytes na aplicação.

Por fim, os testes realizados validaram o funcionamento do sistema, demonstrando sua capacidade de realizar comunicação em tempo real de forma organizada e confiável.

5. Referencias

UNIVERSIDADE DE BRASÍLIA (UnB). Departamento de Ciência da Computação.
Sockets e Concorrência. Material didático da disciplina Redes de Computadores.
Professor: Roberto Rodrigues Filho. Brasília, 2026.

https://docs.python.org/3/library/socket.html

https://www.eletronet.com/blog/entenda-a-diferenca-entre-unicast-multicast-e-broadcast/